

Task 0

I chose to write the algorithm in python, see my submitted .ipynb file for the function.

I copy pasted the decoder array.

```
# Decoder lookup array
decoder = [
    0x00, 0x44, 0x00, 0x54, 0x53, 0x52, 0x48, 0x00, 0x4B,
    0x4C, 0x46, 0x41, 0x00, 0x3F, 0x3E, 0x45, 0x00, 0x01,
    0x02, 0x03, 0x04, 0x05, 0x06, 0x07, 0x08, 0x09, 0x40,
    0x00, 0x49, 0x42, 0x4A, 0x47, 0x51, 0x24, 0x25, 0x26,
    0x27, 0x28, 0x29, 0x2A, 0x2B, 0x2C, 0x2D, 0x2E, 0x2F,
    0x30, 0x31, 0x32, 0x33, 0x34, 0x35, 0x36, 0x37, 0x38,
    0x39, 0x3A, 0x3B, 0x3C, 0x3D, 0x4D, 0x00, 0x4E, 0x43,
    0x00, 0x00, 0x0A, 0x0B, 0x0C, 0x0D, 0x0E, 0x0F, 0x10,
    0x11, 0x12, 0x13, 0x14, 0x15, 0x16, 0x17, 0x18, 0x19,
    0x1A, 0x1B, 0x1C, 0x1D, 0x1E, 0x1F, 0x20, 0x21, 0x22,
    0x23, 0x4F, 0x00, 0x50, 0x00, 0x00
]
```

Here is my function. It takes in the block of 5 chars, converts them to ascii, applies the -32 offset, indexes the lookup table and applies the formula. Then it uses bit shifting and masking to get the 4 decoded bytes out and returns them in an array.

```
# Takes a block of 5 chars, decodes it into 4 chars
def z85_decode_block(chars):
    if len(chars) != 5:
        raise ValueError("Must provide exactly 5 characters")

    value = 0

    # Get the ASCII value, decode using array and apply formula
    for c in chars:
        index = ord(c) - 32
        decoded = decoder[index]
        value = value * 85 + decoded

    # Extract the 4 separate bytes using shifting and masking
    b0 = (value >> 24) & 0xFF
    b1 = (value >> 16) & 0xFF
    b2 = (value >> 8) & 0xFF
    b3 = value & 0xFF

    return [b0, b1, b2, b3]
```

Now let's consider an example. The Z85 encoded string `mOcMhgCH:E` would be decoded like this:

Encoded text	m	O	c	M	h		g	C	H	:	E
Decoded value (DEC)	22	50	12	48	17		16	38	43	64	40
32 bit value	$22*85^4+50*85^3+12*85^2+48*85+17$						$16*85^4+38*85^3+43*85^2+64*85+40$				
	1179210797						858862905				
	0100 0110 0100 1001 0101 0100 0010 1101						0011 0011 0011 0001 0011 0101 0011 1001				
ASCII values	0100 0110	0100 1001	0101 0100	0010 1101			0011 0011	0011 0001	0011 0101	0011 1001	
	70	73	84	45			51	49	53	57	
Decoded string	F	I	T	-			3	1	5	9	

The 3rd line in the steps can be represented as
 $\text{value} = (((c0*85 + c1)*85 + c2)*85 + c3)*85 + c4$, so I used this to compute the 32 bit value in the for loop.

Then here I print the 2 test cases:

```

encoded1 = "mOcMh"
encoded2 = "gCH:E"
decoded_bytes1 = z85_decode_block(encoded1)
decoded_bytes2 = z85_decode_block(encoded2)

print("Decoded bytes 1:", decoded_bytes1)
print("As ASCII 1:", ''.join(chr(b) for b in decoded_bytes1))
print("Decoded bytes 2:", decoded_bytes2)
print("As ASCII 2:", ''.join(chr(b) for b in decoded_bytes2))
✓ 0.0s

Decoded bytes 1: [70, 73, 84, 45]
As ASCII 1: FIT-
Decoded bytes 2: [51, 49, 53, 57]
As ASCII 2: 3159

```

Task 1

Please see my code submission for the full MIPS solution.

```

114 # Get the decoder table address
115 la $t0, decoder
116
117 # Converting the chars to their Z85 values
118 # Char 0
119 addi $t1, $s0, -32
120 add $t1, $t1, $t0
121 lbu $t1, 0($t1)
122
123 # Char 1
124 addi $t2, $s1, -32
125 add $t2, $t2, $t0
126 lbu $t2, 0($t2)
127
128 # Char 2
129 addi $t3, $s2, -32
130 add $t3, $t3, $t0
131 lbu $t3, 0($t3)
132
133 # Char 3
134 addi $t4, $s3, -32
135 add $t4, $t4, $t0
136 lbu $t4, 0($t4)
137
138 # Char 4
139 addi $t5, $s4, -32
140 add $t5, $t5, $t0
141 lbu $t5, 0($t5)

```

In this section I am loading in the decoder table address into \$t0, then going through each loaded character, applying the -32 offset to compute the index in the decoder table and then loading the byte from that index in the table.

```

143 # Building the 32 bit number
144
145 # num = char0, get 85 ready to multiply with
146 move $t6, $t1
147 li $t7, 85
148
149 # num = num*85 + char1
150 mul $t6, $t6, $t7
151 add $t6, $t6, $t2
152
153 # num = num*85 + char2
154 mul $t6, $t6, $t7
155 add $t6, $t6, $t3
156
157 # num = num*85 + char3
158 mul $t6, $t6, $t7
159 add $t6, $t6, $t4
160
161 # num = num*85 + char4
162 mul $t6, $t6, $t7
163 add $t6, $t6, $t5

```

Then here I am iteratively applying the formula for the 32 bit integer same as I did in the python function, multiplying by 85 and adding the next value each time.

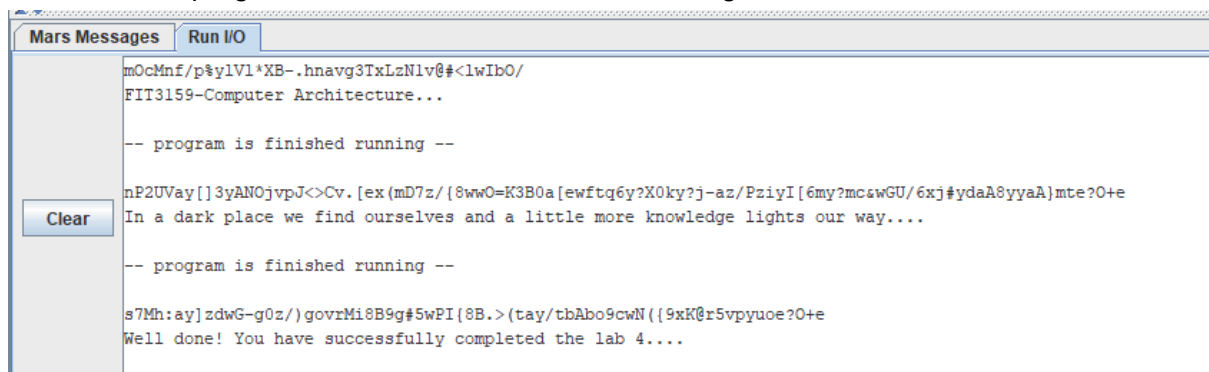
```

165 # Extracting the bytes and printing them
166
167 # byte 0
168 srl $a0, $t6, 24
169 andi $a0, $a0, 0xFF
170 li $v0, 11
171 syscall
172
173 # byte 1
174 srl $a0, $t6, 16
175 andi $a0, $a0, 0xFF
176 li $v0, 11
177 syscall
178
179 # byte 2
180 srl $a0, $t6, 8
181 andi $a0, $a0, 0xFF
182 li $v0, 11
183 syscall
184
185 # byte 3
186 srl $a0, $t6, 0
187 andi $a0, $a0, 0xFF
188 li $v0, 11
189 syscall

```

Then finally I am using shift right logical and and immediate to shift and mask the 32 bit integer to extract the decoded bytes, same as in the python function.

Here I run the program and enter the 3 encoded messages from the lab:



```

Mars Messages Run I/O
mOcMnf/p$ylVl*XB-.hnavg3TxLzNlv@#<lwIbO/
FIT3159-Computer Architecture...

-- program is finished running --

nP2UVay[]3yANOjvpJ<>Cv.[ex(mD7z/{8ww0=K3B0a[ewftq6y?X0ky?j-az/PziyI[6my?mc&wGU/6xj#ydaA8yyaA}mte?O+e
In a dark place we find ourselves and a little more knowledge lights our way....

-- program is finished running --

s7Mh:ay]zdwG-g0z/)govrMi8B9g#5wPI{8B.>(tay/tbAbo9cwN({9xK@r5vpyuoe?O+e
Well done! You have successfully completed the lab 4....

```